Exp 1

1. Fermet Theorem

```python
import random
import math

def fermet_test(n , k):
    if n < 2:
        return False
    if n in (2, 3):
        return True
    if n % 2 == 0:
        return False
    for _ in range(k):
        a = random.randint(2, n-2)
        if math.gcd(a, n) != 1:
            return False
        if pow(a, n-1, n) != 1:
            return False
    return True

n = int(input("Enter a number: "))
k = int(input("Enter number of iteration: "))

if fermet_test(n , k):
    print(n, " is probably prime")
else:
    print(n, " is composite")
```

2. Euler Theorem

```python
import random
import math

def jacobi(a, n):
    result = 1

    while a:
        a %= n
        if a % 2 == 0:
            a //= 2
            if n % 8 in (3, 5):
                result = - result

        a, n = n, a
```

```
        if a % 4 == 3 and n % 4 == 3:
            result = -result

        a %= n
    return result if n == 1 else 0

def euler_test(n, k=5):
    if n < 2:
        return False
    if n == 2 or n == 3:
        return True
    if n % 2 == 0:
        return False
    for i in range(k):
        a = random.randint(2, n-2)
        if math.gcd(a, n) != 1:
            return False

        x = pow(a, (n-1)//2, n)
        j = jacobi(a, n) % n

        if x != j:
            return False
    return True

n = int(input("Enter a number: "))
if euler_test(n):
    print(n, " is probably Prime")
else:
    print(n, " is composite")
```

3. Miller Rabin

```
import random

def miller_rabin(n, k = 5):
    if n < 2:
        return False
    if n == 2 or n == 3:
        return True
    if n % 2 == 0:
        return False

    d = n - 1
    s = 0
    while d % 2 == 0:
        d //= 2
        s += 1
```

```
    for _ in range(k):
        a = random.randint(2, n-2)
        x = pow(a, d, n)

        if x == 1 or x == n-1:
            continue

        for _ in range(s - 1):
            x = pow(x, 2, n)
            if x == n-1:
                break
        else:
            return False
    return True

n = int(input("Enter a number: "))
if miller_rabin(n):
    print(n," is prbably prime")
else:
    print(n, " is composite")
```

4. Euclid

```
def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return abs(a)

a = int(input("Enter value of a: "))
b = int(input("Enter value of b: "))
print("GCD = ",gcd(a,b))
```

```
import math
numbers = list(map(int, input("Enter numbers: ").split()))
result = numbers[0]

for num in numbers[1:]:
    result = math.gcd(result, num)

print("GCD = ",result)
```

```
import math

a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
```

```
g = math.gcd(a, b)

if a == 0 or b == 0:
    lcm = 0
else:
    lcm = abs(a * b) // g

print("GCD =", g)
print("LCM =", lcm)
```

5. Extended Euclid Algorithm

```
def extended_gcd(a, b):
    if b == 0:
        return a, 1, 0

    g, x1, y1 = extended_gcd(b, a % b)

    x = y1
    y = x1 - (a // b) * y1

    return g, x, y

def module_invers(a, m):
    g, x, y = extended_gcd(a, m)

    print("GCD =", g)
    print("x =", x)
    print("y =", y)
    print("Equation:", a, "*", x, "+", m, "*", y, "=", g)

    if g != 1:
        return None
    return x % m

a = int(input("Enter a number: "))
m = int(input("Enter modulus: "))

result = module_invers(a, m)
if result is None:
    print("Modular Inverse does not exists")
else:
    print("Modular Inverse = ",result)
```

6. Euler Totient

```python
def eular_totient(n):
    result = n
    p = 2

    while p*p <= n:
        if n % p == 0:
            while n % p == 0:
                n //= p

            result -= result // p
        p += 1

    if n > 1:
        result -= result // n
    return result

n = int(input("Enter a number: "))
if n < 1:
    print("Enter a positive number!")
else:
    print("Euler Totient = ",eular_totient(n))
```

7. Primitive Root

```python
import math

def primitive_roots(n):
    roots = []

    for g in range(1, n):
        values = set()

        for i in range(1, n + 1):
            values.add(pow(g, i, n))

        if len(values) == sum(
            1 for i in range(1, n + 1)
            if math.gcd(i, n) == 1
        ):
            if math.gcd(g, n) == 1:
                roots.append(g)
    return roots

n = int(input("Enter modulus: "))
roots = primitive_roots(n)
```

```
if roots:
    print("Primitive roots are:", roots)
else:
    print("No Primitive roots exist")
```

8. Discrete Logarithm

```
def discrete_log(g, h, p):
    result = 1

    for x in range(p):
        if result == h:
            return x
        result = (result * g) % p
    return -1

g = int(input("Enter base(g): "))
h = int(input("Enter result(h): "))
p = int(input("Enter modulus(p): "))

x = discrete_log(g, h, p)

if x == -1:
    print("Discrete logarithm does not exist")
else:
    print("Discrete Logarithm = ",x)
```

9. Square and multiply

```
def square_multiply(a, b, n):
    result = 1
    a = a % n

    while b > 0:
        if b % 2 == 1:
            result = (result * a) % n
        a = (a * a) % n
        b = b // 2
    return result

a = int(input("Enter base: "))
b = int(input("Enter exponent: "))
n = int(input("Enter modulus: "))

print("Result = ",square_multiply(a, b, n))
```

Exp 2

1. Caesar Cipher

```
def encrypt(text, key):
    result = ""

    for ch in text:
        if ch.isupper():
            result += chr((ord(ch) - 65 + key) % 26 + 65)
        elif ch.islower():
            result += chr((ord(ch) - 97 + key) % 26 + 97)
        else:
            result += ch
    return result

def decrypt(text, key):
    print("Key used for decryption:", -key)
    return encrypt(text, -key)

text = input("Enter plaintext: ")
key = int(input("Enter Key: "))

cipher = encrypt(text, key)
print("Encrypted Text: ",cipher)

plain = decrypt(cipher, key)
print("Decrypted text: ",plain)
```

```
def encrypt(text, key):
    result = ""

    for ch in text:
        if ch.isupper():
            result += chr((ord(ch) - 65 + key) % 26 + 65)
        elif ch.islower():
            result += chr((ord(ch) - 97 + key) % 26 + 97)
        else:
            result += ch

    return result


def decrypt(text, key):
    return encrypt(text, -key)
```

```
uid = int(input("Enter your UID: "))
text = input("Enter plaintext: ")

key = uid % 26

print("Encryption Key:", key)

cipher = encrypt(text, key)
print("Encrypted Text:", cipher)

plain = decrypt(cipher, key)
print("Decrypted Text:", plain)
```

2. Monoalphabetic Cipher

```
import string

plain = string.ascii_uppercase
cipher = "QWERTYUIOPASDFGHJKLZXCVBNM"

def encrypt(text):
    result = ""
    for ch in text:
        if ch.isupper():
            result += cipher[plain.index(ch)]
        elif ch.islower():
            result += cipher[plain.index(ch.upper())].lower()
        else:
            result += ch
    return result

def decrypt(text):
    result = ""
    for ch in text:
        if ch.isupper():
            result += plain[cipher.index(ch)]
        elif ch.islower():
            result += plain[cipher.index(ch.upper())].lower()
        else:
            result += ch
    return result

text = input("Enter plaintext: ")

encrypted = encrypt(text)
print("Encrypted Text: ",encrypted)

decrypted = decrypt(encrypted)
```

```
print("Decrypted Text: ",decrypted)
```

```
import string

plain = string.ascii_uppercase

def create_key(uid):
    letters = list(plain)
    shift = uid % 26

    cipher = letters[shift:] + letters[:shift]
    return "".join(cipher)


def encrypt(text, cipher):
    result = ""

    for ch in text:
        if ch.isupper():
            result += cipher[plain.index(ch)]
        elif ch.islower():
            result += cipher[plain.index(ch.upper())].lower()
        else:
            result += ch

    return result


def decrypt(text, cipher):
    result = ""

    for ch in text:
        if ch.isupper():
            result += plain[cipher.index(ch)]
        elif ch.islower():
            result += plain[cipher.index(ch.upper())].lower()
        else:
            result += ch

    return result


uid = int(input("Enter your UID: "))
text = input("Enter plaintext: ")

cipher = create_key(uid)
```

```
print("Cipher Key:", cipher)

encrypted = encrypt(text, cipher)
print("Encrypted Text:", encrypted)

decrypted = decrypt(encrypted, cipher)
print("Decrypted Text:", decrypted)
```

3. Vigenere Cipher

```
def encrypt(text, key):
    result = ""
    key = key.upper()
    j = 0

    for ch in text:
        if ch.isalpha():
            shift = ord(key[j % len(key)]) - 65

            if ch.isupper():
                result += chr((ord(ch) - 65 + shift) % 26 + 65)
            else:
                result += chr((ord(ch) - 97 + shift) % 26 + 97)

            j += 1
        else:
            result += ch
    return result

def decrypt(text, key):
    result = ""
    key = key.upper()
    j = 0

    for ch in text:
        if ch.isalpha():
            shift = ord(key[j % len(key)]) - 65

            if ch.isupper():
                result += chr((ord(ch) - 65 - shift) % 26 + 65)
            else:
                result += chr((ord(ch) - 97 - shift) % 26 + 97)
            j += 1
        else:
            result += ch
    return result

text = input("Enter plaintext: ")
```

```
key = input("Enter Key: ")

if not key.isalpha():
    print("Key must contain only letters")
else:
    cipher = encrypt(text, key)
    print("Encrypted Text:", cipher)

    plain = decrypt(cipher, key)
    print("Decrypted Text:", plain)
```

```
def encrypt(text, key):
    result = ""
    key = key.upper()
    j = 0

    for ch in text:
        if ch.isalpha():
            shift = ord(key[j % len(key)]) - 65

            if ch.isupper():
                result += chr((ord(ch) - 65 + shift) % 26 + 65)
            else:
                result += chr((ord(ch) - 97 + shift) % 26 + 97)

            j += 1
        else:
            result += ch

    return result


def decrypt(text, key):
    result = ""
    key = key.upper()
    j = 0

    for ch in text:
        if ch.isalpha():
            shift = ord(key[j % len(key)]) - 65

            if ch.isupper():
                result += chr((ord(ch) - 65 - shift) % 26 + 65)
            else:
                result += chr((ord(ch) - 97 - shift) % 26 + 97)

            j += 1
        else:
```

```
            result += ch

    return result


uid = int(input("Enter your UID: "))

# Generate key using UID
key = chr((uid % 26) + 65)

print("Generated Key:", key)

text = input("Enter plaintext: ")

cipher = encrypt(text, key)
print("Encrypted Text:", cipher)

plain = decrypt(cipher, key)
print("Decrypted Text:", plain)
```

4. Playfair Cipher

```
def create_matrix(key):
    key = key.upper().replace("J", "I")
    letters = ""

    for ch in key + "ABCDEFGHIKLMNOPQRSTUVWXYZ":
        if ch.isalpha() and ch not in letters:
            letters += ch

    return [list(letters[i:i+5]) for i in range(0, 25, 5)]


def prepare(text):
    text = text.upper().replace("J", "I")
    text = "".join(ch for ch in text if ch.isalpha())
    pairs = []
    i = 0

    while i < len(text):
        a = text[i]
        if i + 1 == len(text):
            b = "X" if a != "X" else "Q"
            i += 1
        elif text[i] == text[i+1]:
            b = "X" if a != "X" else "Q"
            i += 1
        else:
```

```
            b = text[i+1]
            i += 2

        pairs.append((a, b))

    return pairs


def playfair(text, matrix, mode):
    result = ""

    for a, b in prepare(text) if mode == "encrypt" else [
        (text[i], text[i+1]) for i in range(0, len(text), 2)
    ]:
        for r in range(5):
            for c in range(5):
                if matrix[r][c] == a:
                    r1, c1 = r, c
                if matrix[r][c] == b:
                    r2, c2 = r, c

        if r1 == r2:
            shift = 1 if mode == "encrypt" else -1
            result += matrix[r1][(c1+shift) % 5]
            result += matrix[r2][(c2+shift) % 5]

        elif c1 == c2:
            shift = 1 if mode == "encrypt" else -1
            result += matrix[(r1+shift) % 5][c1]
            result += matrix[(r2+shift) % 5][c2]

        else:
            result += matrix[r1][c2]
            result += matrix[r2][c1]

    return result


key = input("Enter key: ")
text = input("Enter plaintext: ")

matrix = create_matrix(key)

print("\nPlayfair Matrix:")
for row in matrix:
    print(" ".join(row))

cipher = playfair(text, matrix, "encrypt")
print("\nEncrypted Text:", cipher)
```

```
plain = playfair(cipher, matrix, "decrypt")
print("Decrypted Text:", plain)
```

```
def create_matrix(key):
    key = key.upper().replace("J", "I")
    letters = ""

    for ch in key + "ABCDEFGHIKLMNOPQRSTUVWXYZ":
        if ch.isalpha() and ch not in letters:
            letters += ch

    return [list(letters[i:i+5]) for i in range(0, 25, 5)]


def prepare(text):
    text = text.upper().replace("J", "I")
    text = "".join(ch for ch in text if ch.isalpha())
    pairs = []
    i = 0

    while i < len(text):
        a = text[i]

        if i + 1 == len(text):
            b = "X" if a != "X" else "Q"
            i += 1
        elif text[i] == text[i+1]:
            b = "X" if a != "X" else "Q"
            i += 1
        else:
            b = text[i+1]
            i += 2

        pairs.append((a, b))

    return pairs


def playfair(text, matrix, mode):
    result = ""

    pairs = prepare(text) if mode == "encrypt" else [
        (text[i], text[i+1]) for i in range(0, len(text), 2)
    ]

    for a, b in pairs:
        for r in range(5):
```

```
            for c in range(5):
                if matrix[r][c] == a:
                    r1, c1 = r, c
                if matrix[r][c] == b:
                    r2, c2 = r, c

        if r1 == r2:
            shift = 1 if mode == "encrypt" else -1
            result += matrix[r1][(c1+shift) % 5]
            result += matrix[r2][(c2+shift) % 5]

        elif c1 == c2:
            shift = 1 if mode == "encrypt" else -1
            result += matrix[(r1+shift) % 5][c1]
            result += matrix[(r2+shift) % 5][c2]

        else:
            result += matrix[r1][c2]
            result += matrix[r2][c1]

    return result


# Generate key using UID
uid = int(input("Enter your UID: "))
key = str(uid)

text = input("Enter plaintext: ")

matrix = create_matrix(key)

print("\nPlayfair Matrix:")
for row in matrix:
    print(" ".join(row))

cipher = playfair(text, matrix, "encrypt")
print("\nEncrypted Text:", cipher)

plain = playfair(cipher, matrix, "decrypt")
print("Decrypted Text:", plain)
```

5. Hill Cipher(2*2)

```
def inverse(a, m):
    for i in range(m):
        if (a * i) % m == 1:
            return i
    return None
```

```python
def hill_encrypt(text, key):
    text = text.upper().replace(" ", "")

    if len(text) % 2 != 0:
        text += "X"

    result = ""

    for i in range(0, len(text), 2):
        a = ord(text[i]) - 65
        b = ord(text[i+1]) - 65

        x = (key[0][0]*a + key[0][1]*b) % 26
        y = (key[1][0]*a + key[1][1]*b) % 26

        result += chr(x + 65) + chr(y + 65)

    return result


def hill_decrypt(text, key):
    det = (key[0][0]*key[1][1] - key[0][1]*key[1][0]) % 26
    det_inv = inverse(det, 26)

    if det_inv is None:
        return "Key is not invertible"

    inv = [
        [(key[1][1]*det_inv) % 26, (-key[0][1]*det_inv) % 26],
        [(-key[1][0]*det_inv) % 26, (key[0][0]*det_inv) % 26]
    ]

    result = ""

    for i in range(0, len(text), 2):
        a = ord(text[i]) - 65
        b = ord(text[i+1]) - 65

        x = (inv[0][0]*a + inv[0][1]*b) % 26
        y = (inv[1][0]*a + inv[1][1]*b) % 26

        result += chr(x + 65) + chr(y + 65)

    return result


key = []
```

```
print("Enter 2x2 key matrix:")
for i in range(2):
    row = list(map(int, input().split()))
    key.append(row)

text = input("Enter plaintext: ")

cipher = hill_encrypt(text, key)
print("Encrypted Text:", cipher)

print("Decrypted Text:", hill_decrypt(cipher, key))
```

```
def inverse(a, m):
    for i in range(m):
        if (a * i) % m == 1:
            return i
    return None


def hill_encrypt(text, key):
    text = text.upper().replace(" ", "")

    if len(text) % 2 != 0:
        text += "X"

    result = ""

    for i in range(0, len(text), 2):
        a = ord(text[i]) - 65
        b = ord(text[i+1]) - 65

        x = (key[0][0]*a + key[0][1]*b) % 26
        y = (key[1][0]*a + key[1][1]*b) % 26

        result += chr(x + 65) + chr(y + 65)

    return result


def hill_decrypt(text, key):
    det = (key[0][0]*key[1][1] - key[0][1]*key[1][0]) % 26
    det_inv = inverse(det, 26)

    if det_inv is None:
        return "Key is not invertible"

    inv = [
```

```
        [(key[1][1]*det_inv) % 26, (-key[0][1]*det_inv) % 26],
        [(-key[1][0]*det_inv) % 26, (key[0][0]*det_inv) % 26]
    ]

    result = ""

    for i in range(0, len(text), 2):
        a = ord(text[i]) - 65
        b = ord(text[i+1]) - 65

        x = (inv[0][0]*a + inv[0][1]*b) % 26
        y = (inv[1][0]*a + inv[1][1]*b) % 26

        result += chr(x + 65) + chr(y + 65)

    return result


uid = int(input("Enter your UID: "))

# Generate a valid key matrix from UID
n = uid % 26
key = [[n, 1], [1, 0]]

print("Generated Key Matrix:")
for row in key:
    print(row)

text = input("Enter plaintext: ")

cipher = hill_encrypt(text, key)
print("Encrypted Text:", cipher)

print("Decrypted Text:", hill_decrypt(cipher, key))
```

6. Hill Cipher (3 * 3)

```
def inverse(a):
    for i in range(26):
        if (a * i) % 26 == 1:
            return i
    return -1


def encrypt(text, k):
    text = text.upper().replace(" ", "")
    text += "X" * ((3 - len(text) % 3) % 3)
    result = ""
```

```
    for i in range(0, len(text), 3):
        p = [ord(ch) - 65 for ch in text[i:i+3]]

        for row in k:
            value = sum(row[j] * p[j] for j in range(3)) % 26
            result += chr(value + 65)

    return result


def decrypt(text, k):
    det = (
        k[0][0]*(k[1][1]*k[2][2]-k[1][2]*k[2][1])
        - k[0][1]*(k[1][0]*k[2][2]-k[1][2]*k[2][0])
        + k[0][2]*(k[1][0]*k[2][1]-k[1][1]*k[2][0])
    ) % 26

    d = inverse(det)

    if d == -1:
        return "Invalid Key"

    cof = [
        [k[1][1]*k[2][2]-k[1][2]*k[2][1],
         k[1][2]*k[2][0]-k[1][0]*k[2][2],
         k[1][0]*k[2][1]-k[1][1]*k[2][0]],

        [k[0][2]*k[2][1]-k[0][1]*k[2][2],
         k[0][0]*k[2][2]-k[0][2]*k[2][0],
         k[0][1]*k[2][0]-k[0][0]*k[2][1]],

        [k[0][1]*k[1][2]-k[0][2]*k[1][1],
         k[0][2]*k[1][0]-k[0][0]*k[1][2],
         k[0][0]*k[1][1]-k[0][1]*k[1][0]]
    ]

    adj = [[cof[j][i] for j in range(3)] for i in range(3)]
    inv = [[(adj[i][j]*d) % 26 for j in range(3)] for i in range(3)]

    return encrypt(text, inv)


k = [list(map(int, input().split())) for _ in range(3)]

text = input("Enter plaintext: ")

cipher = encrypt(text, k)
print("Encrypted Text:", cipher)
```

```
print("Decrypted Text:", decrypt(cipher, k))
```

```
def inverse(a):
    for i in range(26):
        if (a * i) % 26 == 1:
            return i
    return -1


def encrypt(text, k):
    text = text.upper().replace(" ", "")
    text += "X" * ((3 - len(text) % 3) % 3)
    result = ""

    for i in range(0, len(text), 3):
        p = [ord(ch) - 65 for ch in text[i:i+3]]

        for row in k:
            value = sum(row[j] * p[j] for j in range(3)) % 26
            result += chr(value + 65)

    return result


def decrypt(text, k):
    det = (
        k[0][0]*(k[1][1]*k[2][2]-k[1][2]*k[2][1])
        - k[0][1]*(k[1][0]*k[2][2]-k[1][2]*k[2][0])
        + k[0][2]*(k[1][0]*k[2][1]-k[1][1]*k[2][0])
    ) % 26

    d = inverse(det)

    if d == -1:
        return "Invalid Key"

    cof = [
        [k[1][1]*k[2][2]-k[1][2]*k[2][1],
         k[1][2]*k[2][0]-k[1][0]*k[2][2],
         k[1][0]*k[2][1]-k[1][1]*k[2][0]],

        [k[0][2]*k[2][1]-k[0][1]*k[2][2],
         k[0][0]*k[2][2]-k[0][2]*k[2][0],
         k[0][1]*k[2][0]-k[0][0]*k[2][1]],

        [k[0][1]*k[1][2]-k[0][2]*k[1][1],
         k[0][2]*k[1][0]-k[0][0]*k[1][2],
         k[0][0]*k[1][1]-k[0][1]*k[1][0]]
```

```
    ]

    adj = [[cof[j][i] for j in range(3)] for i in range(3)]
    inv = [[(adj[i][j]*d) % 26 for j in range(3)] for i in range(3)]

    return encrypt(text, inv)


uid = int(input("Enter UID: "))
n = uid % 26

k = [[n, 1, 0],
     [1, 0, 0],
     [0, 0, 1]]

print("Key Matrix:")
for row in k:
    print(row)

text = input("Enter plaintext: ")

cipher = encrypt(text, k)
print("Encrypted Text:", cipher)
print("Decrypted Text:", decrypt(cipher, k))
```

7. Rail Fence Cipher

```
def encrypt(text, key):
    rail = [""] * key
    row, direction = 0, 1

    for ch in text:
        rail[row] += ch
        if row == 0:
            direction = 1
        elif row == key - 1:
            direction = -1
        row += direction

    return "".join(rail)


def decrypt(text, key):
    pattern = [0] * len(text)
    row, direction = 0, 1

    for i in range(len(text)):
        pattern[i] = row
```

```
        if row == 0:
            direction = 1
        elif row == key - 1:
            direction = -1
        row += direction

    rail = [""] * key
    index = 0

    for r in range(key):
        for i in range(len(text)):
            if pattern[i] == r:
                rail[r] += text[index]
                index += 1

    result = ""
    pos = [0] * key

    for r in pattern:
        result += rail[r][pos[r]]
        pos[r] += 1

    return result


text = input("Enter plaintext: ")
key = int(input("Enter number of rails: "))

cipher = encrypt(text, key)
print("Encrypted Text:", cipher)
print("Decrypted Text:", decrypt(cipher, key))
```

```
def encrypt(text, key):
    rail = [""] * key
    row, direction = 0, 1

    for ch in text:
        rail[row] += ch
        if row == 0:
            direction = 1
        elif row == key - 1:
            direction = -1
        row += direction

    return "".join(rail)


def decrypt(text, key):
```

```
    pattern = [0] * len(text)
    row, direction = 0, 1

    for i in range(len(text)):
        pattern[i] = row
        if row == 0:
            direction = 1
        elif row == key - 1:
            direction = -1
        row += direction

    rail = [""] * key
    index = 0

    for r in range(key):
        for i in range(len(text)):
            if pattern[i] == r:
                rail[r] += text[index]
                index += 1

    result = ""
    pos = [0] * key

    for r in pattern:
        result += rail[r][pos[r]]
        pos[r] += 1

    return result


uid = int(input("Enter UID: "))
key = (uid % 3) + 2

print("Number of Rails:", key)

text = input("Enter plaintext: ")

cipher = encrypt(text, key)
print("Encrypted Text:", cipher)
print("Decrypted Text:", decrypt(cipher, key))
```

8. Columnar transposition

```
def encrypt(text, key):
    n = len(key)
    text = text.upper().replace(" ", "")
    text += "X" * ((n - len(text) % n) % n)
```

```
    rows = [text[i:i+n] for i in range(0, len(text), n)]
    order = sorted(range(n), key=lambda i: key[i])

    return "".join(row[i] for i in order for row in rows)


def decrypt(cipher, key):
    n = len(key)
    rows = len(cipher) // n
    order = sorted(range(n), key=lambda i: key[i])

    matrix = [[""] * n for _ in range(rows)]
    pos = 0

    for col in order:
        for row in range(rows):
            matrix[row][col] = cipher[pos]
            pos += 1

    return "".join("".join(row) for row in matrix)


text = input("Enter plaintext: ")
key = input("Enter numeric key: ")

cipher = encrypt(text, key)
print("Encrypted Text:", cipher)
print("Decrypted Text:", decrypt(cipher, key))
```

```
def encrypt(text, key):
    n = len(key)
    text = text.upper().replace(" ", "")
    text += "X" * ((n - len(text) % n) % n)

    rows = [text[i:i+n] for i in range(0, len(text), n)]
    order = sorted(range(n), key=lambda i: key[i])

    return "".join(row[i] for i in order for row in rows)


def decrypt(cipher, key):
    n = len(key)
    rows = len(cipher) // n
    order = sorted(range(n), key=lambda i: key[i])

    matrix = [[""] * n for _ in range(rows)]
    pos = 0
```

```
    for col in order:
        for row in range(rows):
            matrix[row][col] = cipher[pos]
            pos += 1

    return "".join("".join(row) for row in matrix)


uid = input("Enter UID: ")

# Generate column key using UID
key = "".join(str(int(d) % 9 + 1) for d in uid[:4])

print("Generated Key:", key)

text = input("Enter plaintext: ")

cipher = encrypt(text, key)
print("Encrypted Text:", cipher)
print("Decrypted Text:", decrypt(cipher, key))
```

9. Vernam Cipher / One Time Pad

```
def encrypt(text, key):
    result = ""
    for i in range(len(text)):
        result += chr(ord(text[i]) ^ ord(key[i]))
    return result


text = input("Enter plaintext: ")
key = input("Enter key: ")

if len(text) != len(key):
    print("Key length must equal plaintext length")
else:
    cipher = encrypt(text, key)

    print("Encrypted Text:", cipher.encode().hex().upper())

    plain = encrypt(cipher, key)
    print("Decrypted Text:", plain)
```

```
def encrypt(text, key):
    result = ""
    for i in range(len(text)):
        result += chr(ord(text[i]) ^ ord(key[i]))
```

```
    return result


uid = input("Enter UID: ")
text = input("Enter plaintext: ")

key = (uid * ((len(text) // len(uid)) + 1))[:len(text)]

print("Generated Key:", key)

cipher = encrypt(text, key)
print("Encrypted Text:", cipher.encode().hex().upper())

plain = encrypt(cipher, key)
print("Decrypted Text:", plain)
```

10. Autokey

```
def encrypt(text, key):
    text = text.upper().replace(" ", "")
    key = key.upper() + text
    result = ""

    for i in range(len(text)):
        p = ord(text[i]) - 65
        k = ord(key[i]) - 65
        result += chr((p + k) % 26 + 65)

    return result


def decrypt(cipher, key):
    key = key.upper()
    result = ""

    for i in range(len(cipher)):
        c = ord(cipher[i]) - 65
        k = ord(key[i]) - 65
        p = (c - k) % 26

        result += chr(p + 65)
        key += chr(p + 65)

    return result


text = input("Enter plaintext: ")
key = input("Enter initial key: ")
```

```
cipher = encrypt(text, key)
print("Encrypted Text:", cipher)

plain = decrypt(cipher, key)
print("Decrypted Text:", plain)
```

```
def encrypt(text, key):
    text = text.upper().replace(" ", "")
    key = key.upper() + text
    result = ""

    for i in range(len(text)):
        p = ord(text[i]) - 65
        k = ord(key[i]) - 65
        result += chr((p + k) % 26 + 65)

    return result


def decrypt(cipher, key):
    key = key.upper()
    result = ""

    for i in range(len(cipher)):
        c = ord(cipher[i]) - 65
        k = ord(key[i]) - 65
        p = (c - k) % 26

        result += chr(p + 65)
        key += chr(p + 65)

    return result


uid = input("Enter UID: ")
key = "".join(chr(int(d) % 26 + 65) for d in uid)

text = input("Enter plaintext: ")

print("Generated Key:", key)

cipher = encrypt(text, key)
print("Encrypted Text:", cipher)

plain = decrypt(cipher, key)
print("Decrypted Text:", plain)
```

Exp 3

1. RSA

```
import math

p = int(input("Enter prime p: "))
q = int(input("Enter prime q: "))

n = p * q
phi = (p - 1) * (q - 1)

e = 2
while math.gcd(e, phi) != 1:
    e += 1

d = pow(e, -1, phi)

print("Public Key:", e, n)
print("Private Key:", d, n)

msg = int(input("Enter message (number): "))

cipher = pow(msg, e, n)
print("Encrypted:", cipher)

plain = pow(cipher, d, n)
print("Decrypted:", plain)
```

```
import math

def prime(n):
    while True:
        if n > 1 and all(n % i != 0 for i in range(2, int(n**0.5)+1)):
            return n
        n += 1


uid = int(input("Enter UID: "))

p = prime(uid % 50 + 10)
q = prime((uid // 10) % 50 + 10)

if p == q:
    q = prime(q + 1)
```

```
n = p * q
phi = (p - 1) * (q - 1)

e = 2
while math.gcd(e, phi) != 1:
    e += 1

d = pow(e, -1, phi)

print("Public Key:", e, n)
print("Private Key:", d, n)

msg = int(input("Enter message (number): "))

cipher = pow(msg, e, n)
print("Encrypted:", cipher)

plain = pow(cipher, d, n)
print("Decrypted:", plain)
```

2. Elgamal

```
p = int(input("Enter prime number: "))
g = int(input("Enter primitive root: "))
x = int(input("Enter private key: "))
m = int(input("Enter message (number): "))
k = int(input("Enter random key: "))

# Key generation
y = pow(g, x, p)

# Encryption
c1 = pow(g, k, p)
c2 = (m * pow(y, k, p)) % p

print("Public Key:", y)
print("Ciphertext:", c1, c2)

# Decryption
s = pow(c1, x, p)
plain = (c2 * pow(s, -1, p)) % p

print("Decrypted Message:", plain)
```

```python
uid = int(input("Enter UID: "))

def prime(n):
    while True:
        if n > 1 and all(n % i for i in range(2, int(n**0.5) + 1)):
            return n
        n += 1

p = prime(uid % 50 + 20)
g = 2
x = uid % (p - 2) + 1
m = 10
k = 3

# Key generation
y = pow(g, x, p)

# Encryption
c1 = pow(g, k, p)
c2 = (m * pow(y, k, p)) % p

print("Public Key:", y)
print("Ciphertext:", c1, c2)

# Decryption
s = pow(c1, x, p)
plain = (c2 * pow(s, -1, p)) % p

print("Decrypted Message:", plain)
```

3. Diffie–Hellman key exchange

```python

p = int(input("Enter prime number: "))
g = int(input("Enter generator: "))

a = int(input("Enter Alice's private key: "))
b = int(input("Enter Bob's private key: "))

# Public keys
A = pow(g, a, p)
B = pow(g, b, p)

print("Alice's Public Key:", A)
print("Bob's Public Key:", B)
```

```
# Shared secret keys
key1 = pow(B, a, p)
key2 = pow(A, b, p)

print("Alice's Shared Key:", key1)
print("Bob's Shared Key:", key2)
```

```
uid = int(input("Enter UID: "))

p = 23
g = 5

# Generate private keys
a = uid % 10 + 1
b = (uid // 10) % 10 + 1

# Public keys
A = pow(g, a, p)
B = pow(g, b, p)

print("Alice's Public Key:", A)
print("Bob's Public Key:", B)

# Shared secret keys
key1 = pow(B, a, p)
key2 = pow(A, b, p)

print("Alice's Shared Key:", key1)
print("Bob's Shared Key:", key2)
```

4. ECDH

```
p = 17
a = 2
G = (5, 1)

# Point addition
def add(P, Q):
    if P is None: return Q
    if Q is None: return P

    x1, y1 = P
    x2, y2 = Q
```

```
    if x1 == x2 and (y1 + y2) % p == 0:
        return None

    if P == Q:
        m = (3*x1*x1 + a) * pow(2*y1, -1, p)
    else:
        m = (y2-y1) * pow(x2-x1, -1, p)

    m %= p
    x = (m*m-x1-x2) % p
    y = (m*(x1-x)-y1) % p
    return (x, y)

# Point multiplication
def multiply(k, P):
    R = None
    while k:
        if k % 2:
            R = add(R, P)
        P = add(P, P)
        k //= 2
    return R

alice = int(input("Alice's private key: "))
bob = int(input("Bob's private key: "))

# Public keys
A = multiply(alice, G)
B = multiply(bob, G)

print("Alice's Public Key:", A)
print("Bob's Public Key:", B)

# Shared secret
key1 = multiply(alice, B)
key2 = multiply(bob, A)

print("Alice's Shared Key:", key1)
print("Bob's Shared Key:", key2)
```

```
uid = int(input("Enter UID: "))

p = 17
a = 2
G = (5, 1)
```

```python
def add(P, Q):
    if P is None: return Q
    if Q is None: return P

    x1, y1 = P
    x2, y2 = Q

    if x1 == x2 and (y1+y2) % p == 0:
        return None

    if P == Q:
        m = (3*x1*x1+a) * pow(2*y1, -1, p)
    else:
        m = (y2-y1) * pow(x2-x1, -1, p)

    m %= p
    x = (m*m-x1-x2) % p
    y = (m*(x1-x)-y1) % p
    return (x, y)

def multiply(k, P):
    R = None
    while k:
        if k % 2:
            R = add(R, P)
        P = add(P, P)
        k //= 2
    return R

# UID-based private keys
alice = uid % 18 + 1
bob = (uid // 10) % 18 + 1

A = multiply(alice, G)
B = multiply(bob, G)

print("Alice's Public Key:", A)
print("Bob's Public Key:", B)

key1 = multiply(alice, B)
key2 = multiply(bob, A)

print("Alice's Shared Key:", key1)
print("Bob's Shared Key:", key2)
```

5. EC-ElGamal encryption

```python
p = 17
a = 2
G = (5, 1)

def add(P, Q):
    if P is None: return Q
    if Q is None: return P

    x1, y1 = P
    x2, y2 = Q

    if x1 == x2 and (y1 + y2) % p == 0:
        return None

    if P == Q:
        m = (3*x1*x1 + a) * pow(2*y1, -1, p)
    else:
        m = (y2-y1) * pow(x2-x1, -1, p)

    m %= p
    x = (m*m-x1-x2) % p
    y = (m*(x1-x)-y1) % p
    return (x, y)

def mul(k, P):
    R = None
    while k:
        if k % 2:
            R = add(R, P)
        P = add(P, P)
        k //= 2
    return R

# Key generation
d = int(input("Enter private key: "))
Q = mul(d, G)

# Message as a curve point
M = mul(int(input("Enter message number: ")), G)
k = int(input("Enter random key: "))

# Encryption
C1 = mul(k, G)
C2 = add(M, mul(k, Q))

print("Public Key:", Q)
```

```
print("Ciphertext:", C1, C2)

# Decryption
S = mul(d, C1)
plain = add(C2, (S[0], (-S[1]) % p))

print("Decrypted Message:", plain)
```

```

uid = int(input("Enter UID: "))

p = 17
a = 2
G = (5, 1)

def add(P, Q):
    if P is None: return Q
    if Q is None: return P

    x1, y1 = P
    x2, y2 = Q

    if x1 == x2 and (y1+y2) % p == 0:
        return None

    if P == Q:
        m = (3*x1*x1+a) * pow(2*y1, -1, p)
    else:
        m = (y2-y1) * pow(x2-x1, -1, p)

    m %= p
    x = (m*m-x1-x2) % p
    y = (m*(x1-x)-y1) % p
    return (x, y)

def mul(k, P):
    R = None
    while k:
        if k % 2:
            R = add(R, P)
        P = add(P, P)
        k //= 2
    return R

# UID-based keys
d = uid % 18 + 1
k = (uid // 10) % 18 + 1
```

```
m = 3

Q = mul(d, G)
M = mul(m, G)

# Encryption
C1 = mul(k, G)
C2 = add(M, mul(k, Q))

print("Public Key:", Q)
print("Ciphertext:", C1, C2)

# Decryption
S = mul(d, C1)
plain = add(C2, (S[0], (-S[1]) % p))

print("Decrypted Message:", plain)
```

Exp 4

1. RSA digital signature

```
import math

p = int(input("Enter prime p: "))
q = int(input("Enter prime q: "))

n = p * q
phi = (p-1) * (q-1)

e = 2
while math.gcd(e, phi) != 1:
    e += 1

d = pow(e, -1, phi)

msg = int(input("Enter message (number): "))

# Signature generation
signature = pow(msg, d, n)
print("Digital Signature:", signature)

# Signature verification
verify = pow(signature, e, n)
```

```
print("Verified Message:", verify)

if verify == msg:
    print("Signature is Valid")
else:
    print("Signature is Invalid")
```

```
import math

uid = int(input("Enter UID: "))

def prime(n):
    while True:
        if n > 1 and all(n % i for i in range(2, int(n**0.5)+1)):
            return n
        n += 1

p = prime(uid % 50 + 10)
q = prime((uid // 10) % 50 + 10)

if p == q:
    q = prime(q + 1)

n = p * q
phi = (p-1) * (q-1)

e = 2
while math.gcd(e, phi) != 1:
    e += 1

d = pow(e, -1, phi)

msg = int(input("Enter message (number): "))

# Generate signature
signature = pow(msg, d, n)
print("Digital Signature:", signature)

# Verify signature
verify = pow(signature, e, n)

print("Verified Message:", verify)

if verify == msg:
    print("Signature is Valid")
```

```
else:
    print("Signature is Invalid")
```

2. DSA

```
import hashlib

p, q, g = 23, 11, 4
x = 6                   # Private key
k = 3                   # Random secret number

msg = input("Enter message: ")

# Hash the message
H = int(hashlib.sha256(msg.encode()).hexdigest(), 16) % q

# Public key
y = pow(g, x, p)

# Signature generation
r = pow(g, k, p) % q
s = (pow(k, -1, q) * (H + x*r)) % q

print("Public Key:", y)
print("Digital Signature:", r, s)

# Signature verification
w = pow(s, -1, q)
u1 = (H*w) % q
u2 = (r*w) % q
v = (pow(g, u1, p) * pow(y, u2, p) % p) % q

print("Signature is Valid" if v == r else "Signature is Invalid")
```

```
import hashlib

uid = int(input("Enter UID: "))

p, q, g = 23, 11, 4

x = uid % 10 + 1
msg = input("Enter message: ")
```

```
H = int(hashlib.sha256(msg.encode()).hexdigest(), 16) % q
y = pow(g, x, p)

# Generate a valid signature
for k in range(2, q):
    r = pow(g, k, p) % q
    if r == 0:
        continue

    s = (pow(k, -1, q) * (H + x*r)) % q
    if s != 0:
        break

print("Public Key:", y)
print("Digital Signature:", r, s)

# Verify signature
w = pow(s, -1, q)
u1 = H*w % q
u2 = r*w % q
v = (pow(g, u1, p) * pow(y, u2, p) % p) % q

print("Signature is Valid" if v == r else "Signature is Invalid")
```

3. ECDSA

```
import hashlib

p, a, n = 17, 2, 19
G = (5, 1)

def add(P, Q):
    if P is None: return Q
    if Q is None: return P

    x1, y1 = P
    x2, y2 = Q

    if x1 == x2 and (y1+y2) % p == 0:
        return None

    if P == Q:
        m = (3*x1*x1+a) * pow(2*y1, -1, p)
    else:
        m = (y2-y1) * pow(x2-x1, -1, p)
```

```
    m %= p
    x = (m*m-x1-x2) % p
    y = (m*(x1-x)-y1) % p
    return x, y

def mul(k, P):
    R = None
    while k:
        if k % 2:
            R = add(R, P)
        P = add(P, P)
        k //= 2
    return R

msg = input("Enter message: ")
d = int(input("Enter private key: "))
k = int(input("Enter random key: "))

H = int(hashlib.sha256(msg.encode()).hexdigest(), 16) % n

# Key generation
Q = mul(d, G)

# Signature generation
R = mul(k, G)
r = R[0] % n
s = (pow(k, -1, n) * (H+d*r)) % n

print("Signature:", r, s)

# Signature verification
w = pow(s, -1, n)
u1 = H*w % n
u2 = r*w % n
X = add(mul(u1, G), mul(u2, Q))

if X is not None and X[0] % n == r:
    print("Signature is Valid")
else:
    print("Signature is Invalid")
```

```
import hashlib

uid = int(input("Enter UID: "))

p, a, n = 17, 2, 19
```

```python
G = (5, 1)

def add(P, Q):
    if P is None: return Q
    if Q is None: return P

    x1, y1 = P
    x2, y2 = Q

    if x1 == x2 and (y1+y2) % p == 0:
        return None

    if P == Q:
        m = (3*x1*x1+a) * pow(2*y1, -1, p)
    else:
        m = (y2-y1) * pow(x2-x1, -1, p)

    m %= p
    x = (m*m-x1-x2) % p
    y = (m*(x1-x)-y1) % p
    return x, y

def mul(k, P):
    R = None
    while k:
        if k % 2:
            R = add(R, P)
        P = add(P, P)
        k //= 2
    return R

msg = input("Enter message: ")

# UID-based keys
d = uid % 18 + 1
k = (uid // 10) % 18 + 1

H = int(hashlib.sha256(msg.encode()).hexdigest(), 16) % n

# Public key
Q = mul(d, G)

# Generate valid signature
for i in range(18):
    R = mul((k+i-1) % 18 + 1, G)
    r = R[0] % n

    if r == 0:
```

```
        continue

    s = (pow((k+i-1) % 18 + 1, -1, n) * (H+d*r)) % n

    if s != 0:
        break

print("Public Key:", Q)
print("Digital Signature:", r, s)

# Verification
w = pow(s, -1, n)
u1 = H*w % n
u2 = r*w % n
X = add(mul(u1, G), mul(u2, Q))

if X is not None and X[0] % n == r:
    print("Signature is Valid")
else:
    print("Signature is Invalid")
```

4. ElGamal digital signature

```
import hashlib
import math

p = int(input("Enter prime number: "))
g = int(input("Enter generator: "))
x = int(input("Enter private key: "))
k = int(input("Enter random key: "))

msg = input("Enter message: ")
H = int(hashlib.sha256(msg.encode()).hexdigest(), 16)

# Key generation
y = pow(g, x, p)

# Signature generation
r = pow(g, k, p)
s = (pow(k, -1, p-1) * (H-x*r)) % (p-1)

print("Public Key:", y)
print("Digital Signature:", r, s)

# Signature verification
v1 = pow(g, H, p)
```

```
v2 = (pow(y, r, p) * pow(r, s, p)) % p

if v1 == v2:
    print("Signature is Valid")
else:
    print("Signature is Invalid")
```

```
import hashlib
import math

uid = int(input("Enter UID: "))

p = 23
g = 5
x = uid % 20 + 1

msg = input("Enter message: ")
H = int(hashlib.sha256(msg.encode()).hexdigest(), 16)

# Generate valid random key
k = uid % 22 + 1
while math.gcd(k, p-1) != 1:
    k = k % 22 + 1

# Public key
y = pow(g, x, p)

# Signature generation
r = pow(g, k, p)
s = (pow(k, -1, p-1) * (H-x*r)) % (p-1)

print("Public Key:", y)
print("Digital Signature:", r, s)

# Verification
v1 = pow(g, H, p)
v2 = (pow(y, r, p) * pow(r, s, p)) % p

print("Signature is Valid" if v1 == v2 else "Signature is Invalid")
```